NumPy Arrays in N Dimensions

Boolean arrays across several axes, turning a mask into coordinates, indexing with integer arrays on more than one axis, counting occurrences, and sorting and searching.

01 Indexing and slicing three axes
02 Boolean arrays on several axes
03 Coordinates from a boolean array
04 Integer arrays on several axes
05 Counting occurrences
06 Sorting and searching
07 Bounding box from a boolean array
08 Loop forms and array equivalents

Scope. Advanced indexing on a single axis is assumed, and so are shape, `axis=` and broadcasting. Locating and counting the elements that satisfy a condition applies all of these on several axes at once.

After reading it you should be able to do the following without looking anything up.

- Build a mask over an array of any rank and reduce it along a chosen axis.
- Convert a mask into coordinates, and say what shape each conversion returns.
- Index with one integer array per axis, and say why the result is a copy.
- Count occurrences with `np.unique` and `np.bincount`, and choose between them.
- Sort, obtain the sorting permutation, and search a sorted array.

Every example assumes `import numpy as np`.

01

Indexing and slicing three axes

Indexing a three-axis array works as it does for one or two. The comma-separated form takes one entry per axis.

A voxel is one cell of a three-dimensional grid, the equivalent of a pixel with a third coordinate. The volume below holds one block id per voxel.

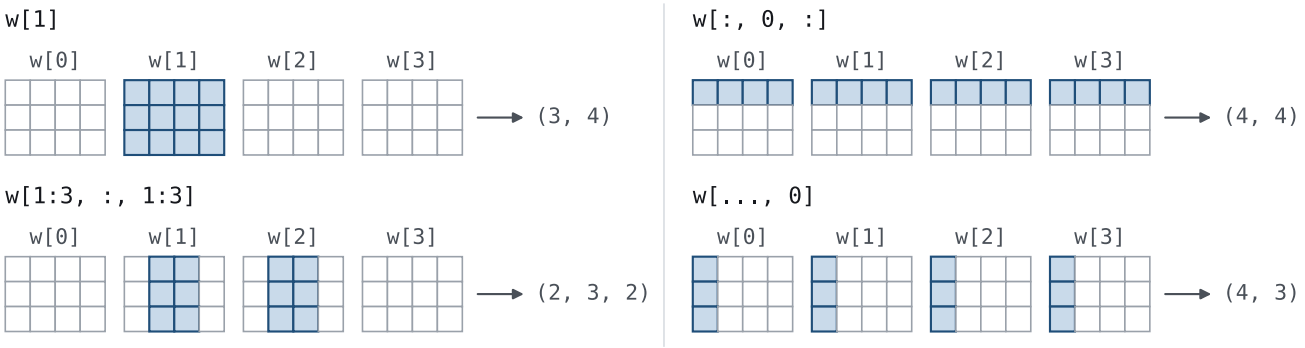
Code comments throughout give shapes, as `(going in) -> (coming out)`. `()` is NumPy's shape for a single value, and `n` stands for however many elements a mask happens to select.

```
w = np.zeros((96, 48, 96), dtype=np.uint8)    # x, y, z

# One block id per voxel, 0 to 13. The ids used in the examples:
AIR, WATER, IRON, GOLD, OBSIDIAN, BEDROCK = 0, 4, 9, 10, 12, 13
N_BLOCK_IDS = 14

w[10, 20, 30]         # (96, 48, 96) -> ()            one element
w[10]                 # (96, 48, 96) -> (48, 96)      axis 0 indexed away
w[:, 0, :]            # (96, 48, 96) -> (96, 96)      one horizontal layer
w[8:12, :, 8:12]      # (96, 48, 96) -> (4, 48, 4)    a column of the volume
w[..., 0]             # (96, 48, 96) -> (96, 48)      same as w[:, :, 0]
```

A single index removes an axis. A slice keeps it. A slice is still a view at any rank, so writing through one changes the volume.



The same four kinds of index on a (4, 3, 4) array w, drawn as its slices w[0] to w[3] along axis 0, each with axis 1 down and axis 2 across. Filled cells are the ones selected.

The meaning of each axis is a convention chosen for the data; NumPy does not record it. If `w` is indexed `[x, y, z]` with `y` as height, then a height map over the ground plane is a reduction along axis 1, and it has shape `(96, 96)`.

02

Boolean arrays on several axes

A comparison on an array of any rank gives a boolean array of the same shape. `sum`, `any` and `all` reduce it along a chosen axis.

```
# w: (96, 48, 96) of block ids

solid = (w != AIR)              # (96, 48, 96) -> (96, 48, 96) bool

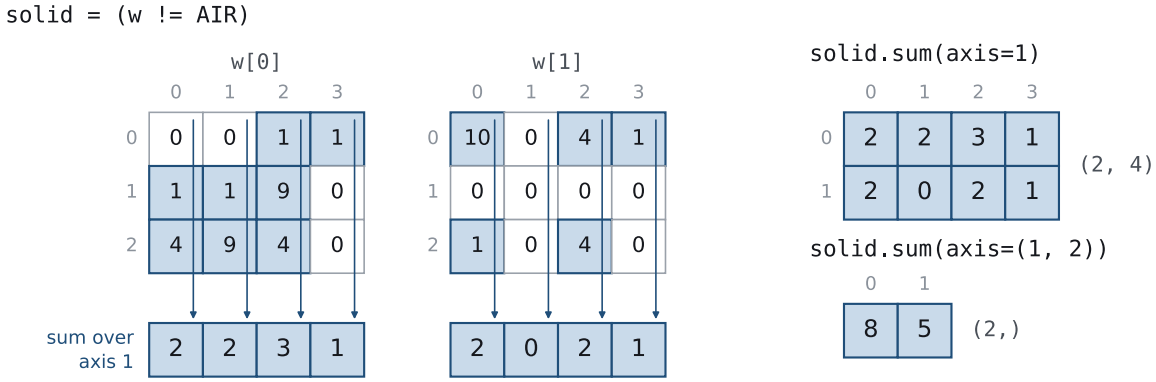
solid.sum()                     # (96, 48, 96) -> ()          how many solid voxels
solid.sum(axis=1)               # (96, 48, 96) -> (96, 96)    a count per ground column
solid.any(axis=1)               # (96, 48, 96) -> (96, 96)    which columns hold anything
solid.all(axis=1)               # (96, 48, 96) -> (96, 96)    which are solid throughout
```

`True` counts as 1, so `sum` on a mask is a count and `mean` is a proportion. The axis rule is unchanged: the named axis is removed from the result shape.

Several axes can be reduced at once by passing a tuple:

```
# solid: (96, 48, 96) bool, True where the voxel is not air

solid.sum(axis=(1, 2))          # (96, 48, 96) -> (96,)       one count per x slab
```



A (2, 3, 4) volume w of block ids, drawn as its slices with axis 1 down and axis 2 across; filled cells are solid. Summing over axis 1 counts down each column and gives one row of the (2, 4) result per slice. Summing over axes 1 and 2 gives one count per slice, (2,).

COMBINING CONDITIONS

Conditions are combined with `&`, `|` and `~`. Python's `and` and `or` raise on an array of more than one element. `&`, `|` and `~` bind more tightly than the comparison operators, so each comparison needs its own brackets.

```
# w: (96, 48, 96) of block ids

ore = (w == IRON) | (w == GOLD)       # (96, 48, 96) -> (96, 48, 96) bool
deep = np.zeros(w.shape, dtype=bool)  # (96, 48, 96) bool
deep[:, :24, :] = True                # the lower half, by height

(ore & deep).sum()                    # (96, 48, 96) -> ()   ore below mid-height
(ore & ~deep).sum()                   # (96, 48, 96) -> ()   ore above it
```

03

Coordinates from a boolean array

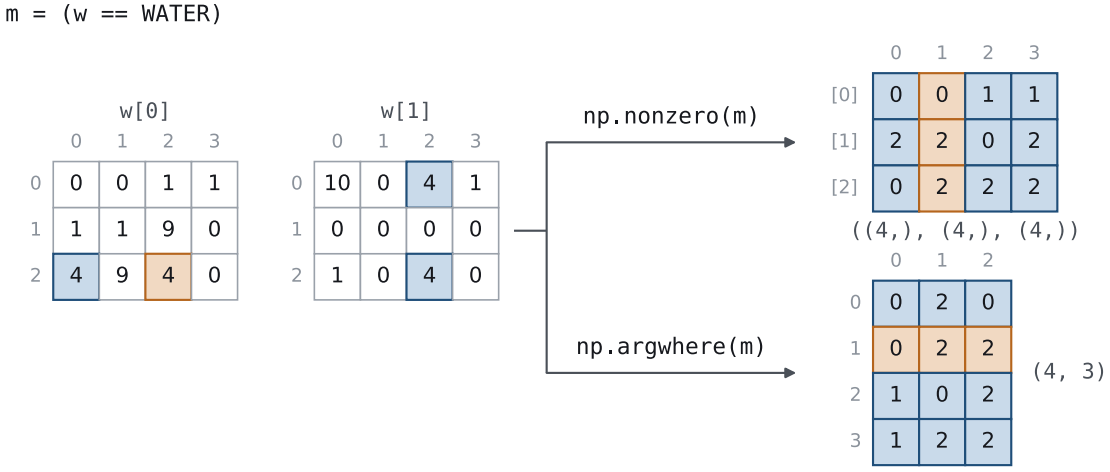
`np.nonzero` and `np.argwhere` both turn a mask into the positions of its `True` values. They return different shapes.

```
# w: (96, 48, 96) of block ids

m = (w == WATER)                # (96, 48, 96) -> (96, 48, 96) bool, n True

np.nonzero(m)                   # (96, 48, 96) -> ((n,), (n,), (n,))
np.argwhere(m)                  # (96, 48, 96) -> (n, 3)
```

`np.nonzero` returns coordinates grouped **by axis**, which is the form needed to index with, so `w[np.nonzero(m)]` works and equals `w[m]`. `np.argwhere` returns them grouped **by point**, one row per match, which is the form that supports iterating over matches and reducing along axis 0.



The 4 water voxels of the (2, 3, 4) volume and their coordinates. np.nonzero gives one array per axis; np.argwhere gives one row per point, (4, 3). The voxel at (0, 2, 2) is drawn in orange in all three places.

```
# m: (96, 48, 96) bool, n of them True

xs, ys, zs = np.nonzero(m)      # (n,), (n,), (n,)
xs.min(), xs.max()              # (n,) -> (), ()        the extent along axis 0

pts = np.argwhere(m)            # (96, 48, 96) -> (n, 3)
pts.min(axis=0)                 # (n, 3) -> (3,)        one corner of the bounding box
pts.max(axis=0)                 # (n, 3) -> (3,)        the opposite corner
pts.mean(axis=0)                # (n, 3) -> (3,)        the centroid
```

Both return empty results rather than raising when nothing matches, so `np.argwhere(m)[0]` raises `IndexError` on no match, and `pts.min(axis=0)` raises on an empty array. `m.any()` is `False` in that case, so it can be tested before either result is indexed.

On a one-dimensional array `np.nonzero` still returns a tuple, which is why `np.nonzero(mask)[0]` has the trailing index, and why `np.flatnonzero` exists as the flat-array shorthand.

04

Integer arrays on several axes

Indexing with one integer array per axis selects a list of points, not a rectangular block.

```
# w: (96, 48, 96) of block ids

xs = np.array([0, 1, 2])       # (3,)
ys = np.array([5, 6, 7])       # (3,)
zs = np.array([9, 9, 9])       # (3,)

w[xs, ys, zs]                  # (96, 48, 96) -> (3,)   one value per index triple
```

The integer arrays are matched up element by element, so they must have the same shape, or broadcast to one. The result has the shape of the broadcast integer arrays, not the shape of the array being indexed. Indexing this way turns a set of coordinates, such as the output of `np.argwhere`, back into values.

```
# w: (96, 48, 96) of block ids    m: (96, 48, 96) bool, n True

pts = np.argwhere(m)                          # (96, 48, 96) -> (n, 3)
vals = w[pts[:, 0], pts[:, 1], pts[:, 2]]     # three (n,) indexers -> (n,)
vals = w[tuple(pts.T)]                        # the same, written shorter -> (n,)
```

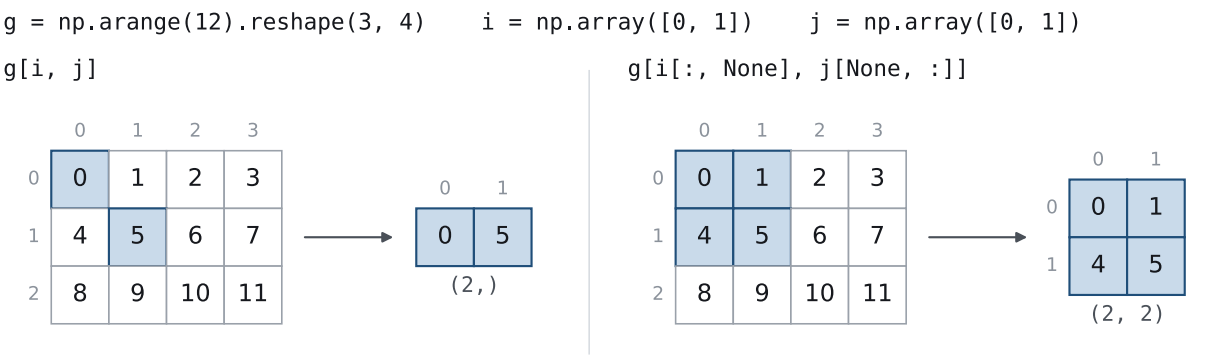
POSITION OF THE BROADCAST AXES

Mixing slices with integer arrays is allowed. The integer arrays broadcast together into one block of axes, and the position of that block in the result depends on where the integer arrays appear in the index. When the integer arrays are **next to each other**, the block goes where they were. When a slice **separates** them, there is no single position for the block, and NumPy puts it first.

```
x = np.zeros((2, 3, 4, 5))     # (2, 3, 4, 5)
i = np.array([0, 1])           # (2,)
j = np.array([0, 1])           # (2,)   broadcasts with i to (2,)

# i and j are zipped, not crossed: the pairs are (0, 0) and (1, 1), so 2 points,
# not 2 x 2 = 4. Axes 1 and 2 are consumed together and replaced by a single axis,
# whose length is the broadcast shape of i and j.
x[:, i, j, :]                  # (2, 3, 4, 5) -> (2, 2, 5)   block stays in place
x[i, :, j, :]                  # (2, 3, 4, 5) -> (2, 3, 5)   block moves to the front

# For all 2 x 2 combinations, i and j are broadcast explicitly. A (2,1) against a
# (1,2) gives a (2, 2) index grid, and that grid becomes two axes in the result.
x[:, i[:, None], j[None, :], :]   # (2,1) v (1,2) -> (2,2)   so -> (2, 2, 2, 5)
```



The same i and j on two axes of a (3, 4) array g. g[i, j] pairs them element by element and selects the 2 points (0, 0) and (1, 1). Broadcast as (2, 1) against (1, 2), they select all 4 combinations, as a (2, 2) result.

The extra axes in the result are the shape of the **broadcast indexers**, never one axis per indexer. Two `(2,)` arrays broadcast to `(2,)` and contribute one axis. A `(2, 1)` and a `(1, 2)` broadcast to `(2, 2)` and contribute two.

In the first line above, the `(2,)` block replaces axes 1 and 2 where they stood, leaving `(2, 2, 5)`. In the second, axes 0 and 2 are indexed and axis 1 remains between them, so the block goes to the front and axis 1 follows it. Taking the rectangular slice first and indexing that avoids the issue.

> **COPIES FROM ADVANCED INDEXING**
>
> Basic slicing always returns a view. Advanced indexing, by integer array or by boolean array, always returns a copy. The chosen positions are arbitrary, so no single stride per axis can describe them, and the data has to be gathered into a new array.
>
> ```
> a = np.array([10, 20, 30, 40, 50])   # (5,)
>
> a[[0, 1]]                      # (5,) -> (2,)   .base is None, a new array
> a[a > 15]                      # (5,) -> (4,)   .base is None, a new array
> a[0:2]                         # (5,) -> (2,)   .base is a,    a view
> ```
>
> Assignment through them still writes into the original array. `w[m] = 0` clears every matching voxel.

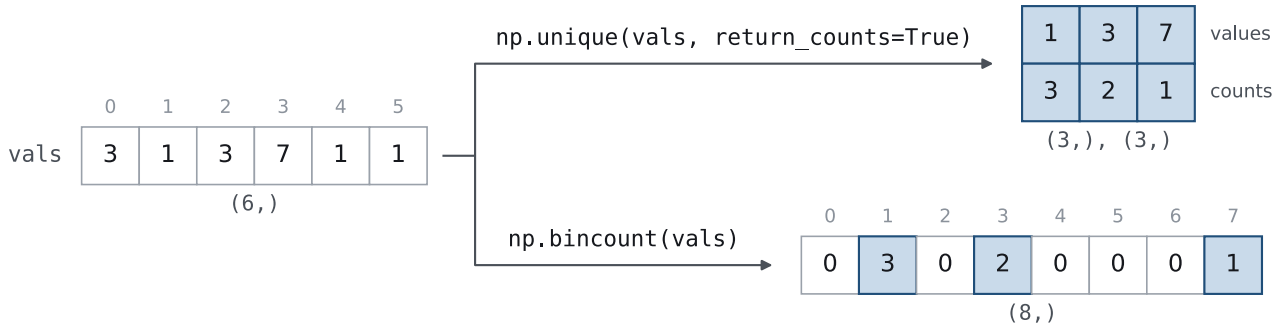
05

Counting occurrences

`np.unique` and `np.bincount` both count occurrences. They suit different data.

```
vals = np.array([3, 1, 3, 7, 1, 1])      # (6,)

np.unique(vals)                          # (6,) -> (3,)         [1 3 7]
np.unique(vals, return_counts=True)      # (6,) -> (3,), (3,)   values, then counts
np.bincount(vals)                        # (6,) -> (8,)         0 to max, by value
```



np.unique returns the 3 distinct values and their counts. np.bincount returns one count for every value from 0 to the maximum, 7, so its result has shape (8,) and is indexed by the value itself.

`np.unique` works on any values and returns only the ones that occur, paired with their counts. `np.bincount` takes non-negative integers and returns one entry per value from 0 up to the maximum, whether or not it occurs, so its output is indexed by the value itself. `np.bincount(vals)[3]` is how many threes there are.

Counting how many voxels hold each of the fourteen ids shows the difference. One mask per id gives the counts, but each mask is a full pass over the volume, so fourteen ids means fourteen passes:

```
# w: (96, 48, 96) of block ids 0 to 13    N_BLOCK_IDS = 14

counts = [(w == i).sum() for i in range(N_BLOCK_IDS)]        # 14 passes -> list of 14
counts = np.bincount(w.reshape(-1), minlength=N_BLOCK_IDS)   # 1 pass     -> (14,)
counts[IRON]                                                 # (14,) -> ()  iron ore
```

`minlength` sets a minimum length for the result, so every id has an entry even when a high id does not occur. Without it the array stops at the largest id present, and `counts[BEDROCK]` would raise `IndexError` on a volume with no bedrock in it.

`np.unique` takes `return_index` for the first position of each distinct value, and `return_inverse` for the map that rebuilds the original from the unique values. The latter is how a column of labels becomes integer codes.

06

Sorting and searching

Sorting the values, obtaining the permutation that sorts them, and locating a value in a sorted array without scanning it.

```
s = np.array([5, 1, 4, 2])   # (4,)

np.sort(s)                # (4,) -> (4,)   [1 2 4 5], a sorted copy
np.argsort(s)             # (4,) -> (4,)   [1 3 2 0], the sorting indices
s.sort()                  # in place, returns None
```

`np.argsort` returns indices rather than values, so the same permutation can reorder any array of the same length. `values[np.argsort(keys)]` orders the values by their keys. With `axis=` both sort along one axis of a larger array.

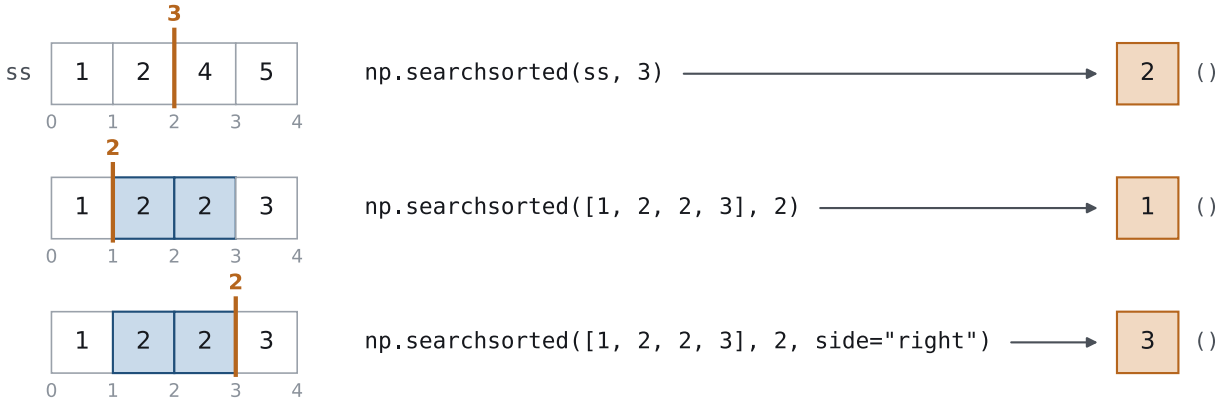
SEARCHSORTED

Given a sorted array, `np.searchsorted` returns the position where a value would be inserted to keep it sorted, by binary search, in `O(log n)`. It accepts an array of values and returns one position per value.

```
# s: (4,) = [5 1 4 2]

ss = np.sort(s)                                  # (4,)  [1 2 4 5]
np.searchsorted(ss, 3)                           # (4,), scalar -> ()   2

np.searchsorted([1, 2, 2, 3], 2)                 # -> ()   1, the first position
np.searchsorted([1, 2, 2, 3], 2, side="right")   # -> ()   3, one past the last
```



np.searchsorted returns the position, from 0 to the length of the array, where the value would be inserted to keep the array sorted; the positions are numbered under the cells. With side="right" it is the position after the last equal element, so the two answers for 2 differ by its 2 copies.

The gap between the two sides is the number of copies of that value, which is another way to count occurrences in sorted data.

COUNTING SMALLER ELEMENTS

Counting, for each element, how many others are smaller than it can be written three ways, costing `O(n²)` time, `O(n²)` memory and `O(n log n)` respectively.

```
a = np.array([8, 1, 2, 2, 3])                # (n,)
n = len(a)

count = [0] * n
for i in range(n):                           # a Python loop, O(n^2) time
    count[i] = sum(1 for x in a if x < a[i])

(a[:, None] > a[None, :]).sum(axis=1)        # (n,1) v (1,n) -> (n,n) -> (n,)

np.searchsorted(np.sort(a), a)               # (n,) -> (n,)   O(n log n), no square
```

All three give the same counts, also for repeated values such as the two 2s in `a`. The third sorts once and then binary-searches every element, so it neither loops in Python nor allocates an `n` by `n` array. At `n = 100_000` the middle version allocates 10 GB and the third allocates one array of `n` integers.

07

Bounding box from a boolean array

A bounding box comes from the coordinates of a mask: the smallest and largest coordinate on each axis are the two opposite corners of the box that contains every match, and slicing the volume between them keeps only

that box.

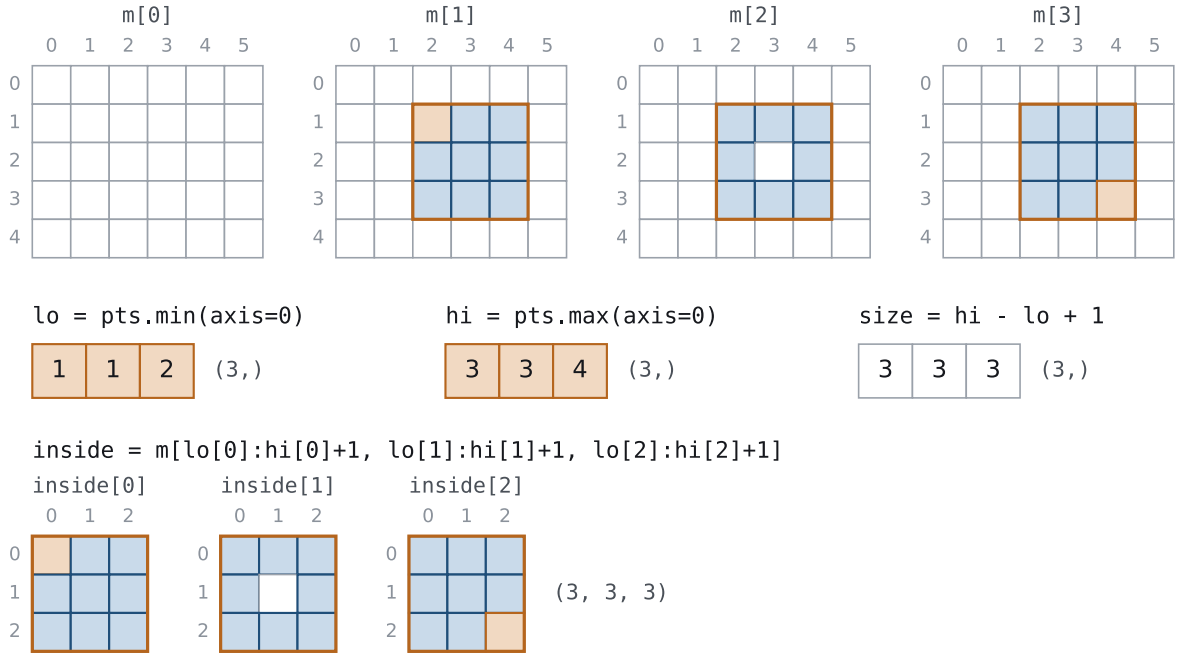
```
# w: (96, 48, 96) of block ids    OBSIDIAN = 12    N_BLOCK_IDS = 14
# The vault is a hollow cube of obsidian, buried somewhere in the volume.

# 1. build the mask
m = (w == OBSIDIAN)
if not m.any():
    raise ValueError("no candidates")

# 2. turn the mask into points
pts = np.argwhere(m)               # (96, 48, 96) -> (n, 3)

# 3. measure the points, not the volume
lo = pts.min(axis=0)               # (n, 3) -> (3,)   one corner of the bounding box
hi = pts.max(axis=0)               # (n, 3) -> (3,)   the opposite corner
size = hi - lo + 1                 # (3,)   -> (3,)   extent per axis, inclusive
centre = (lo + hi) // 2            # (3,)   -> (3,)   integer centre

# 4. count the block ids inside the box only
inside = w[lo[0]:hi[0]+1, lo[1]:hi[1]+1, lo[2]:hi[2]+1]
np.bincount(inside.reshape(-1), minlength=N_BLOCK_IDS)
```



The steps on a (4, 5, 6) mask m holding a hollow 3 by 3 by 3 cube, drawn as its slices m[0] to m[3]. pts = np.argwhere(m) has shape (26, 3). lo and hi, the smallest and largest coordinate on each axis, are the corners drawn in orange. Slicing from lo to hi + 1 on every axis keeps the outlined box, inside, of shape (3, 3, 3).

After step 2 the mask has become an `(n, 3)` array of points, so every measurement in step 3 is a reduction along axis 0 of that array. The volume itself is not read again until step 4.

`min` and `max` return occupied coordinates and a slice upper bound is exclusive, so both `size` and the slice bounds need the `+ 1`.

08

Loop forms and array equivalents

Each expression in the right-hand column runs in compiled code and references no index variable.

WHAT THE LOOP DOES	AS AN ARRAY EXPRESSION
Count elements satisfying a condition	`(arr > 2).sum()`
Count per row of a 2D array	`(arr > 2).sum(axis=1)`
Count over two axes at once	`mask.sum(axis=(1, 2))`
Positions of every match, by axis	`np.nonzero(mask)`
Positions of every match, one row each	`np.argwhere(mask)`
Bounding box of the matches	`pts.min(axis=0)`, `pts.max(axis=0)`
Values at a list of coordinates	`arr[tuple(pts.T)]`
Set every match to zero	`arr[mask] = 0`
How many of each id	`np.bincount(arr.reshape(-1), minlength=k)`
Distinct values and their counts	`np.unique(arr, return_counts=True)`
Check for duplicates	`len(np.unique(a)) < len(a)`
Order one array by another	`vals[np.argsort(keys)]`
How many are smaller than each element	`np.searchsorted(np.sort(a), a)`

FURTHER READING

- Indexing on ndarrays. The advanced indexing section covers integer array indexing, boolean array indexing, and their combination with slices.
- Sorting, searching and counting, the reference for `np.sort`, `np.argsort`, `np.searchsorted`, `np.nonzero` and `np.argwhere`.
- Set routines, for `np.unique`, `np.isin`, `np.intersect1d` and the other set operations.
- The routine reference, every NumPy function grouped by topic.